BellaStyle Manager

Sistema de gestión para salón de belleza

Diagrama de Componentes

Versión con interfaces (sin flechas) y cómo explicarla

César Pestana
Proyecto de Aula · Ingeniería de Sistemas
Fundación Universitaria Colombo Internacional
Cartagena de Indias, octubre de 2026 · Versión 3.0

Contenido

1. Qué está pidiendo el profesor

2. Vista general

3. Vista detallada

3.1 Interfaces del diagrama

4. Cómo explicarlo en la sustentación

4.1 Guion de un minuto

4.2 Preguntas probables

1. Qué está pidiendo el profesor

El profesor observó que en el diagrama de componentes **no se deben unir los componentes con flechas**. Tiene razón en la forma que se enseña en clase: en un diagrama de componentes, los componentes se comunican **a través de interfaces**, no con flechas directas.

La idea es esta: un componente es como una "caja negra". Desde afuera no importa cómo está hecho por dentro, solo importa **qué servicio ofrece** y **qué servicio necesita**. Eso se dibuja con dos símbolos:

Símbolo	Nombre	Significado
○ (bola)	Interfaz provista	El componente **ofrece** un servicio. Ejemplo: el Backend ofrece la API REST.
◖ (media luna)	Interfaz requerida	El componente **necesita** un servicio. Ejemplo: el Frontend necesita la API REST.
○ dentro de ◖	Conector de ensamble	Un componente necesita lo que el otro ofrece: así se **comunican**.
Rectángulo con icono	Componente	Una parte del sistema que se puede reemplazar (un módulo, un archivo, una librería).
Rectángulo con pestaña	Paquete	Agrupa componentes (Frontend y Backend).

Analogía: es como un enchufe. El tomacorriente de la pared **provee** energía (bola) y la clavija del televisor la **requiere** (media luna). Al conectarlos encajan. Nadie dibuja una flecha del televisor a la pared: se dibuja el enchufe.

Por qué la versión anterior tenía flechas: UML 2.5 sí permite flechas punteadas de dependencia (<<use>>). Pero el estilo que pide el profesor es el "puro", donde **toda** la comunicación pasa por interfaces. Por eso se hizo esta versión nueva, **sin ninguna flecha**. La versión anterior se conserva en la carpeta `diagramas`.

2. Vista general

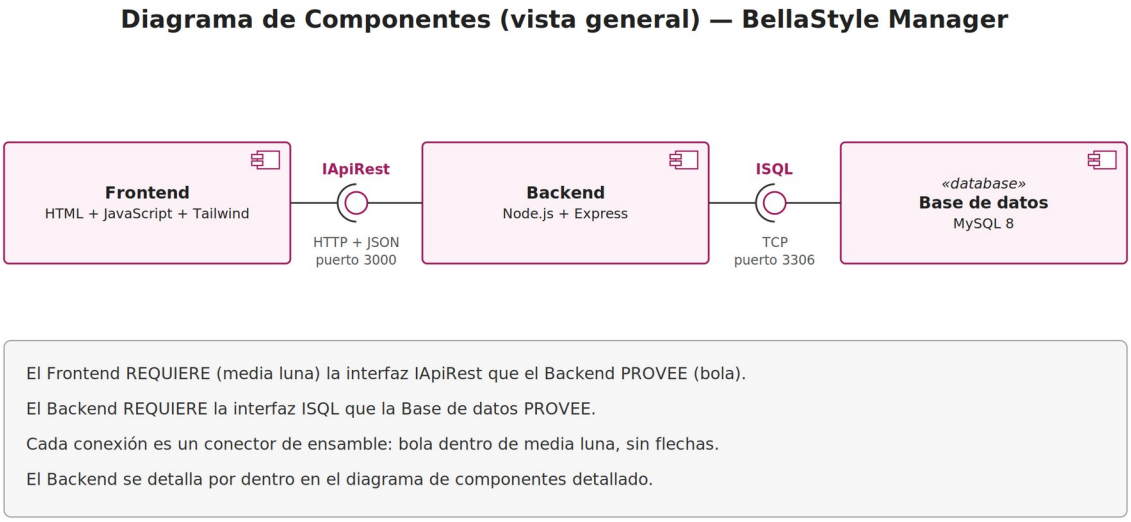


Figura 1. Vista general: tres componentes unidos por interfaces

Es la forma más simple de presentar el sistema. Conviene mostrarla primero:

- El **Frontend** requiere la interfaz **IApiRest**, que el **Backend** provee (HTTP + JSON, puerto 3000).
- El **Backend** requiere la interfaz **ISQL**, que la **Base de datos** provee (puerto 3306).

3. Vista detallada

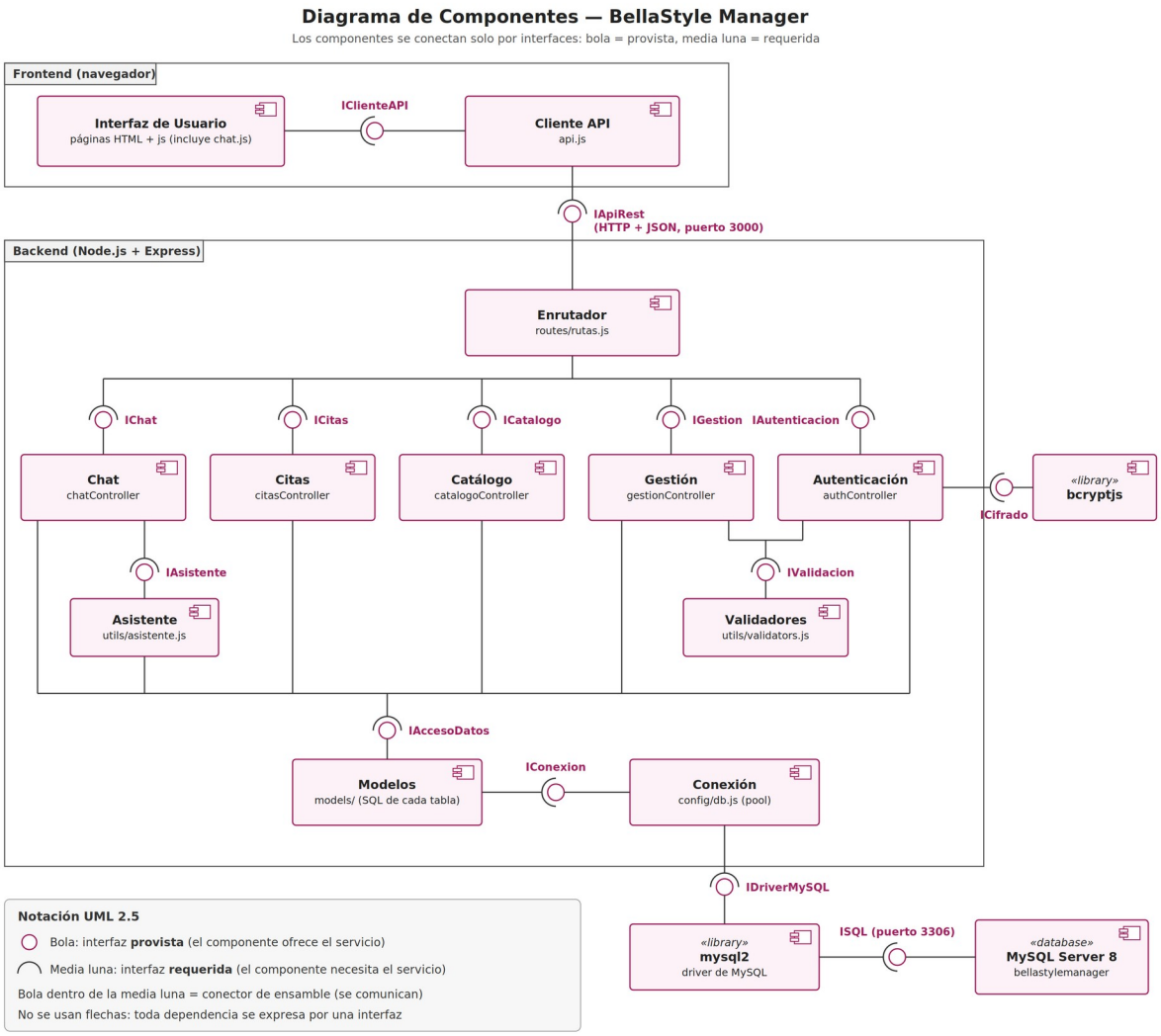


Figura 2. Vista detallada: el interior del Frontend y el Backend

3.1 Interfaces del diagrama

Cada interfaz tiene quien la **provee** (bola) y quien la **requiere** (media luna):

Interfaz	La provee	La requiere	Qué servicio es
IClienteAPI	Cliente API (api.js)	Interfaz de Usuario	Enviar peticiones y manejar la sesión.
IApiRest	Enrutador (rutas.js)	Cliente API	Las rutas /api por HTTP, puerto 3000.
IChat	chatController	Enrutador	Chat de la página de inicio: mensajes del visitante y respuestas.
IAsistente	Asistente (asistente.js)	Chat	Respuestas automáticas a preguntas frecuentes.
ICitas	citasController	Enrutador	Listar, crear, reprogramar y cambiar estado de citas.

Interfaz	La provee	La requiere	Qué servicio es
ICatalogo	catalogoController	Enrutador	Consultar servicios, productos, estilistas y clientas.
IGestion	gestionController	Enrutador	Crear, editar, activar y desactivar.
IAutenticacion	authController	Enrutador	Login, registro y recuperar contraseña.
IValidacion	Validadores	Gestión y Autenticación	Revisar nombres, usuarios y contraseñas.
ICifrado	bcryptjs	Autenticación	Cifrar y comparar contraseñas.
IAccesoDatos	Modelos	Los 5 controladores y el Asistente	Consultas SQL de cada tabla.
IConexion	Conexión (db.js)	Modelos	El pool de conexiones a MySQL.
IDriverMySQL	mysql2	Conexión	Hablar el protocolo de MySQL.
ISQL	MySQL Server	mysql2	Ejecutar SQL, puerto 3306.

Nota: cuando una media luna recibe varias líneas (por ejemplo, IAccesoDatos), significa que **varios componentes requieren la misma interfaz**. Es válido en UML.

4. Cómo explicarlo en la sustentación

4.1 Guion de un minuto

“Este es el diagrama de componentes de BellaStyle Manager. Los componentes no se unen con flechas, sino con **interfaces**: la bola es lo que un componente **ofrece** y la media luna lo que **necesita**.

En la vista general hay tres componentes: el Frontend requiere la API REST que provee el Backend, y el Backend requiere el acceso SQL que provee MySQL.

En la vista detallada, el Enrutador provee la API y requiere cinco interfaces, una por cada controlador: chat, citas, catálogo, gestión y autenticación. Los controladores requieren IAccesoDatos, que proveen los Modelos, y los Modelos requieren la conexión, que usa la librería mysql2 para llegar a MySQL. Así cada parte se puede cambiar sin afectar a las demás, siempre que mantenga su interfaz.”

4.2 Preguntas probables

Pregunta	Respuesta corta
¿Qué es una interfaz?	Un contrato: el conjunto de servicios que un componente ofrece, sin mostrar cómo los hace por dentro.
¿Diferencia entre provista y requerida?	Provista (bola) = lo que el componente ofrece. Requerida (media luna) = lo que necesita de otro.
¿Qué es un conector de ensamble?	La unión de una media luna con una bola: indica que un componente usa el servicio del otro.
¿Por qué no hay flechas?	Porque en este diagrama toda comunicación se expresa por interfaces; así se ve qué servicio exacto une a cada par de componentes.
¿Qué ventaja tiene?	Bajo acoplamiento: si se cambia MySQL por otra base, solo cambia quien provee ISQL; el resto no se toca.
¿Diferencia con el de despliegue?	Componentes muestra QUÉ partes lógicas hay y cómo se comunican; despliegue muestra DÓNDE corren físicamente.